

Discrete Flow Matching for Antibody Sequence Generation

Theoretical Framework, Scorer Design, and Research Directions

Internal Working Document

May 13, 2026

Contents

1	Motivation and Problem Setting	1
2	Background: From Continuous to Discrete Diffusion	2
2.1	Continuous diffusion models	2
2.2	Why discrete diffusion for protein sequences	2
3	Discrete Flow Matching: Theory	2
3.1	From vector fields to jump rate matrices	2
3.2	Constructing the probability path	3
3.3	Training objective	3
3.4	Source distribution choices	3
3.5	Position independence and epistasis	4
4	Biasing Generation Toward Developability	4
4.1	Posterior tempering	4
4.2	Comparison with SVDD	5
5	Scorer Design Options	5
5.1	Level 0: ESM-2 embeddings + shallow regressor on GDPa (baseline)	5
5.2	Level 1: Physics-informed sequence features	5
5.3	Level 2: SSH2.0 pseudo-labels on OAS (core novel contribution)	6
5.4	Level 3 (ablation): Deep ensemble with uncertainty quantification	7
5.5	Level 4: AbLang-2 naturalness regularizer	7
5.6	Summary of scorer options	8
6	Generative Backbone: Implementation Considerations	8
6.1	Choice of backbone	8
6.2	OAS data pipeline	8
7	Pareto Optimization: Considerations for Future Work	8
7.1	Why Pareto matters	9
7.2	Integration points	9
7.3	Uncertainty and the Pareto front	9
7.4	Scientific value	9
8	Recommended Research Roadmap	9

Abstract

This document lays out the theoretical and practical foundations for a research project aimed at generating therapeutic antibody sequences with improved developability profiles. We motivate the use of **discrete flow matching** as the generative backbone, derive the core mathematics of the framework, explain how external scorer models can bias generation toward desirable biophysical properties, survey the available scorer options at increasing levels of scientific novelty, and close with considerations on multi-objective Pareto optimization. The document is intended as a working reference to help the team converge on precise methodological choices before committing to implementation.

1 Motivation and Problem Setting

Therapeutic antibodies must satisfy two largely independent criteria: high-affinity target binding, and *developability*, i.e., the collection of biophysical properties that determine whether a molecule can be manufactured, formulated, and safely administered at scale. Developability encompasses aggregation propensity, colloidal and thermal stability, hydrophobicity, self-interaction, and immunogenicity risk.

Computational antibody design has historically optimized binding through structure-based docking and directed evolution simulations, but developability has largely been treated as a post-hoc filter rather than a design objective. The consequence is high attrition late in the development pipeline.

The goal of this project is to build a generative framework that jointly samples sequences from the natural antibody distribution *while biasing* those samples toward regions of sequence space with favorable developability scores. The key challenges are:

1. The sequence space is discrete (amino acid alphabet $|\mathcal{A}| = 20$), making standard continuous diffusion inapplicable without approximation.
2. Labeled developability data is scarce: the most comprehensive public dataset (GDPa [2]) covers 246 paired heavy-light chain antibodies.
3. Guidance must preserve naturalness; sequences that optimize a predicted score but leave the natural antibody distribution are not therapeutically relevant.

A recent paper by Zhao et al. [1] addressed a closely related problem using a masked discrete diffusion model trained on the Observed Antibody Space (OAS) [3] combined with Soft Value-based Decoding in Diffusion (SVDD) [4] for guided sampling. The present project aims to explore whether replacing SVDD with **discrete flow matching and posterior-tempered classifier guidance** offers a more principled and data-efficient alternative, and to systematically investigate the scorer design space.

2 Background: From Continuous to Discrete Diffusion

2.1 Continuous diffusion models

A continuous diffusion model defines a forward process that gradually corrupts a data sample $x_1 \sim p_{\text{data}}$ into Gaussian noise $x_0 \sim \mathcal{N}(0, I)$ via an SDE, and learns a reverse process (score function or denoiser) to invert it. Guidance toward a reward $s(x_1)$ can be introduced through classifier guidance, which requires differentiable proxy models, or through fine-tuning approaches such as classifier-free guidance or RL-based methods, which are computationally expensive.

2.2 Why discrete diffusion for protein sequences

Protein sequences live in $\mathcal{A}^L$ where $\mathcal{A}$ is the amino acid alphabet and L is sequence length. There is no meaningful Euclidean metric on this space: a one-step change from **ALA** to **TRP** is no more or less disruptive than any other single substitution. Applying continuous diffusion requires embedding sequences into a continuous latent space, which introduces a mismatch between the generative process and the actual optimization domain.

Discrete diffusion [5] operates directly on token sequences by defining a Markov chain over $\mathcal{A}^L$. Two canonical noise processes exist:

- **Uniform diffusion:** tokens transition to any other token with equal probability, yielding a uniform marginal at full corruption.
- **Absorbing (masked) diffusion:** tokens are replaced by a special [MASK] token; the model learns to unmask sequences conditioned on context. This is the approach used by Zhao et al. and is equivalent to order-agnostic masked language modeling (OADM) [6].

Absorbing diffusion is well-suited to sequence inpainting (e.g., CDR loop redesign) but has the limitation that at high corruption levels all sequence identity is lost, making it difficult to condition generation on a starting antibody template. **Germline-absorbing diffusion** [7] addresses this by using the germline sequence as the absorbing state, so that intermediate noisy states are always biologically valid antibody sequences at varying degrees of somatic hypermutation.

3 Discrete Flow Matching: Theory

3.1 From vector fields to jump rate matrices

Continuous flow matching [8] learns a vector field $v_t : \mathbb{R}^d \rightarrow \mathbb{R}^d$ that transports p_0 to p_1 along deterministic or stochastic trajectories. For discrete sequences, the analogous object is a **rate matrix** $R_t \in \mathbb{R}^{|\mathcal{A}| \times |\mathcal{A}|}$ governing a continuous-time Markov chain (CTMC). For a single sequence position with current token x , the probability of jumping to token $y \neq x$ in a small interval dt is

$$p(x \rightarrow y \text{ in } [t, t+dt]) = R_t(x, y) dt. \quad (1)$$

Diagonal entries enforce the row-sum-zero constraint: $R_t(x, x) = -\sum_{y \neq x} R_t(x, y)$. The marginal distribution $p_t(x)$ evolves according to the Kolmogorov forward equation:

$$\frac{d}{dt} p_t(x) = \sum_y p_t(y) R_t(y, x). \quad (2)$$

The goal of discrete flow matching is to **learn** R_t such that p_0 is a chosen source distribution and $p_1 = p_{\text{data}}$ (natural antibody sequences).

3.2 Constructing the probability path

Following Campbell et al. [9], one first defines a **conditional path** $p_t(x | x_1)$ for each data sample x_1 , then marginalizes:

$$p_t(x) = \int p_t(x | x_1) p_{\text{data}}(x_1) dx_1. \quad (3)$$

The standard choice is the **linear interpolant**:

$$p_t(x \mid x_1) = (1 - t) p_0(x) + t \delta(x, x_1), \quad (4)$$

which places full mass on x_1 at $t = 1$ and recovers p_0 at $t = 0$. The conditional rate matrix that generates this path is:

$$R_t(x, y \mid x_1) = \frac{\dot{p}_t(y \mid x_1)}{p_t(x \mid x_1)} \cdot \mathbf{1}[y \neq x], \quad (5)$$

and the marginal rate follows by importance weighting over the posterior:

$$R_t(x, y) = \int \frac{p_t(x \mid x_1)}{p_t(x)} R_t(x, y \mid x_1) p_{\text{data}}(x_1 \mid x, t) dx_1. \quad (6)$$

The key object in Eq. (6) is $p_{\text{data}}(x_1 \mid x_t, t)$: the posterior over clean sequences given the noisy state at time t . This is precisely what a trained denoiser predicts.

3.3 Training objective

A neural network $f_\theta(x_t, t)$ is trained to predict the **clean sequence** x_1 given a noisy intermediate x_t . The loss is cross-entropy over sequence positions:

$$\mathcal{L}(\theta) = \mathbb{E}_{t, x_1 \sim p_{\text{data}}, x_t \sim p_t(\cdot \mid x_1)} [-\log f_\theta(x_1 \mid x_t, t)]. \quad (7)$$

At inference, f_θ is substituted into Eq. (6) to obtain $\hat{R}_t$, and the CTMC is simulated from $t = 0$ to $t = 1$ using Euler steps or exact simulation.

3.4 Source distribution choices

The source p_0 is a **design choice** with significant downstream effects:

Uniform over tokens

Maximally uninformative. Intermediates are biologically uninterpretable; the network carries the full burden of imposing antibody structure.

Masked absorbing state

$p_0 = [\text{MASK}]^L$. Recovers masked diffusion as a special case of discrete flow matching.

Germline sequences

$p_0 = p_{\text{germline}}$. Intermediates are valid antibody sequences at varying somatic hypermutation levels, keeping the trajectory on the data manifold throughout. This is biologically principled and provides strong inductive bias.

Empirical low-developability distribution

$p_0 = p_{\text{low-dev}}$. Directly models the transport from poorly developable to well-developable sequences. Requires labeled data to define but is the closest analogue to Schrödinger Bridge semantics.

Remark 1. *The germline-absorbing source is the recommended default for this project: it encodes real antibody biology into the noise process at no additional computational cost, and it ensures that guided generation stays within the antibody sequence distribution even under aggressive property optimization.*

3.5 Position independence and epistasis

Most implementations treat sequence positions as **conditionally independent** given (x_t, t) , so the rate matrix factorizes position-wise. The neural network introduces inter-position correlations through its attention mechanism when predicting $\hat{x}_1$, but the CTMC simulation treats positions independently at each step. This is an approximation: antibody CDR residues exhibit strong epistatic coupling, and a mutation tolerated at one position may be destabilizing in a different sequence context. Fully joint CTMC simulation over $\mathcal{A}^L$ is combinatorially intractable; addressing epistasis in discrete flow matching is an active research direction.

4 Biasing Generation Toward Developability

4.1 Posterior tempering

Because discrete flow matching produces a **predicted clean sequence** $\hat{x}_1 = f_\theta(x_t, t)$ at every step, guidance can be applied directly in token probability space rather than in a noisy latent. Given a developability scorer $s : \mathcal{A}^L \rightarrow \mathbb{R}$, the predicted distribution is reweighted at each step:

$$\tilde{p}(x_1 | x_t, t) \propto f_\theta(x_1 | x_t, t) \cdot \exp(\lambda s(x_1)), \quad (8)$$

where $\lambda > 0$ controls guidance strength. This reweighted distribution is then substituted back into Eq. (6) to compute $\hat{R}_t$, biasing the jumps at each step toward tokens consistent with high-scoring completions.

Three properties of Eq. (8) are practically important:

1. **No differentiability required.** s can be any black-box function (SVM, random forest, physics calculator), unlike classifier guidance in continuous diffusion.
2. **No lookahead sampling.** Unlike SVDD, which draws M candidate states at each step to estimate a value function, posterior tempering intervenes directly on the denoiser output. The computational overhead is one scorer call per step rather than M forward passes.
3. **Tunable naturalness-developability tradeoff.** Adding the log-likelihood under an antibody language model p_{AbLang} as a regularization term:

$$s_{\text{composite}}(x_1) = s_{\text{dev}}(x_1) + \alpha \log p_{\text{AbLang}}(x_1), \quad (9)$$

gives a single scalar α that interpolates between pure developability optimization ($\alpha = 0$) and preservation of naturalness ($\alpha \rightarrow \infty$). Sweeping α empirically characterizes the tradeoff, which is a scientifically novel contribution in its own right.

4.2 Comparison with SVDD

SVDD [4] integrates *soft value functions* into the sampling procedure of a pre-trained diffusion model. At each denoising step, M candidate next states are sampled and scored; the state with the highest value is selected. The value function is estimated by training a separate network on rollout data from the frozen generative model. This is effective but introduces two costs: (i) M -fold increase in network evaluations per step, and (ii) value function training on top of the already scarce labeled data. Posterior tempering in discrete flow matching avoids both by intervening on the denoiser’s own output distribution, at the cost of assuming the scorer is well-calibrated at the token level rather than at the trajectory level.

5 Scorer Design Options

We organize the scorer options in order of increasing scientific contribution. All options are sequence-in, score-out and compatible with Eq. (8). None require proprietary software.

5.1 Level 0: ESM-2 embeddings + shallow regressor on GDPa (baseline)

What it is. Freeze a pretrained ESM-2 [10] model, extract the mean-pooled representation of the concatenated heavy-light chain sequence, and fit a shallow MLP or linear regressor on the 246 GDPa experimental measurements. One model per property (HIC retention time, AC-SINS, etc.).

Scientific role. This is the direct comparison point with Zhao et al. and establishes that any improvement in the present work is attributable to the generative framework or scorer design, not to additional data.

Implementation notes. ESM-2 is available via HuggingFace (`facebook/esm2_t33_650M_UR50D`). The 650M parameter model requires approximately 3 GB of VRAM for inference. Embeddings for 246 sequences can be extracted in minutes and cached on disk; the regressor training is then a CPU-only operation. Cross-validation strategy is important at $N = 246$: use leave-one-out or repeated 5-fold to obtain reliable generalization estimates.

External dependencies (beyond Python packages). HuggingFace model weights are downloaded automatically on first use (~ 2.5 GB for ESM-2 650M). No storage constraints beyond a standard research allocation.

5.2 Level 1: Physics-informed sequence features

What it is. Compute sequence-derived biophysical descriptors that encode domain knowledge about antibody developability, and use them as input features to the GDPa regressor, either in place of or concatenated with ESM-2 embeddings.

Two complementary feature sets are relevant:

TAP (Therapeutic Antibody Profiler) [11]

Computes five developability metrics: CDR length, Protein Surface Hydrophobicity (PSH), Patches of Positive Charge (PPC), Patches of Negative Charge (PNC), and Structural Fv Charge Symmetry Parameter (SFvCSP). TAP internally uses ABodyBuilder2 [12] for structure prediction, making it sequence-in at the API level. ABodyBuilder2 is available via the ImmuneBuilder Python package. Query time is approximately 2–5 seconds per sequence on GPU, making TAP suitable as a terminal scorer on final candidates but *not* for inline guidance at every sampling step.

PROPERMAB [13]

An open framework that trains regularized linear models to predict structure-based descriptors (including SAP, a widely validated aggregation propensity score) directly from sequence, with minimal accuracy loss compared to structure-derived features. PROPERMAB-derived features are near-instantaneous to compute and suitable for inline guidance.

Scientific contribution. Physics-informed features encode prior knowledge that reduces the effective complexity of the regression problem. Using them as auxiliary inputs to a GDPa-trained regressor is a form of physics-informed machine learning that should improve generalization from 246 labeled points, and provides an interpretable ablation.

Implementation notes. The `ImmuneBuilder` package is pip-installable and requires approximately 4 GB of VRAM. `ABodyBuilder2` model weights (~200 MB) are downloaded on first use. For PROPERMAB features, the `iFeatureOmega` Python package provides all required sequence descriptors. TAP’s five subscores can either be computed by reimplementing the geometry from the `ABodyBuilder2` structure output (straightforward but requiring 2–3 weeks of careful engineering) or by querying the TAP web server (<https://opig.stats.ox.ac.uk/webapps/tap>) for the small GDPa set as a one-time preprocessing step.

External dependencies. `ImmuneBuilder` (pip), `iFeatureOmega` (pip). No significant storage requirements beyond the `ABodyBuilder2` weights.

5.3 Level 2: SSH2.0 pseudo-labels on OAS (core novel contribution)

What it is. SSH2.0 [14] is an SVM ensemble trained on 131 antibodies that jointly predicts hydrophobic interaction risk across SMAC, SGAC-SINS, and HIC from sequence alone, achieving 83.97% accuracy with no structure required. The key observation is that SSH2.0 can be *run at the scale of OAS* (millions of sequences) to generate weak binary labels, which can then be used to pretrain a regressor before fine-tuning on the 246 GDPa continuous measurements.

The two-stage training procedure is:

1. Retrain SSH2.0 locally using CKSAAGP features (available via `iFeatureOmega` or `pfeature`) on the published 131-sequence dataset. This gives a locally callable scorer with sub-millisecond query time.
2. Run the local SSH2.0 replica on a 100k–500k subset of OAS paired sequences to generate pseudo-labels (soft probabilities).
3. Pretrain an ESM-2-based regressor on the pseudo-labeled corpus using binary cross-entropy.
4. Swap the output head and fine-tune with MSE on GDPa continuous measurements.

Scientific contribution. This is a **weak supervision** approach to the small-data problem that has not previously been applied in the antibody design literature. The SSL/transfer learning framing is well-motivated and the improvement over Level 0 is directly testable.

Implementation notes. The SSH2.0 web server (<http://i.uestc.edu.cn/SSH2/>) is publicly accessible but is rate-limited and fragile for batch use; local retraining is strongly recommended. The OAS database [3] is available via a public API and download portal; a 100k paired sequence subset requires approximately 2–5 GB of disk space after preprocessing. The full OAS database is substantially larger (hundreds of gigabytes); only a representative subset is needed. The `oas_utils` Python package supports structured queries to avoid downloading the full database.

External dependencies. `iFeatureOmega` or `pfeature` (pip) for CKSAAGP features. OAS data download (2–5 GB for a usable subset). No IP or licensing restrictions; OAS and the SSH2.0 training data are both openly published.

5.4 Level 3 (ablation): Deep ensemble with uncertainty quantification

What it is. Train $N = 5$ independent regressors (differing in random seed or feature subset) and use ensemble variance as an uncertainty estimate. Substitute the **lower confidence bound** (mean minus $\beta \cdot \text{std}$) for guidance to prevent exploitation of high-variance extrapolation regions:

$$s_{\text{LCB}}(x_1) = \mu_{\theta}(x_1) - \beta \sigma_{\theta}(x_1). \quad (10)$$

Scientific role. Uncertainty-aware guidance is a principled response to the small-data setting that reviewers will expect to see addressed. It connects the work to the broader literature on uncertainty-aware molecular optimization and is also the natural foundation for an active learning loop.

Implementation notes. N times the base scorer query time; with fast regressors this remains well under 100 ms per sequence. No additional external dependencies.

5.5 Level 4: AbLang-2 naturalness regularizer

What it is. AbLang-2 [15] is a masked language model trained on paired heavy-light chain sequences from OAS. The **pseudo-log-likelihood** (PLL) of a sequence under AbLang-2, computed as the sum of per-token log probabilities under masking, measures how natural a sequence appears relative to the immune repertoire. This is used as the regularization term $\alpha \log p_{\text{AbLang}}(x_1)$ in Eq. (9).

Scientific contribution. The naturalness-developability tradeoff is a genuine open question in therapeutic antibody design. Characterizing this tradeoff empirically as α varies, and showing that the framework can navigate it in a controlled way, is a clean and publishable contribution independent of the generative model choice.

Implementation notes. AbLang-2 is pip-installable (`pip install ablang2`). A single forward pass for a paired (heavy, light) sequence takes approximately 30–80 ms on GPU and 200–500 ms on CPU, suitable for inline guidance. Model weights are downloaded automatically on first use (~ 400 MB). Usage:

```
import ablang2
model = ablang2.pretrained(model_to_use='ablang2-paired')
pll = model.pseudo_log_likelihood(
    [('EVQLVESGG...', 'DIQMTQSPS...')],
    mode='paired'
)
```

External dependencies. `ablang2` (pip). Model weights ~ 400 MB. No licensing restrictions.

5.6 Summary of scorer options

Level	Scorer	Labeled data	Inline?	Storage	Novel contribution
0	ESM-2 + MLP (GDPa)	246 pts	Yes	~ 2.5 GB	Baseline / replication
1	TAP / PROPERMAB	None	Partial	~ 200 MB	Physics-informed
2	SSH2.0 $\rightarrow$ GDPa	OAS subset	Yes	2–5 GB	Weak supervision at scale
3	Deep ensemble	Same as above	Yes	Same	Uncertainty-aware guidance
4	AbLang-2 PLL	None (pretrained)	Yes	~ 400 MB	Naturalness–dev tradeoff

Table 1: Scorer options summary. “Inline” refers to suitability for guidance at every sampling step (query time < 100 ms). TAP is marked partial because its structure prediction step (~ 3 s) makes it suitable as a terminal evaluator only.

6 Generative Backbone: Implementation Considerations

6.1 Choice of backbone

Two options are available for the discrete flow matching backbone:

Retrain from OAS

Train an antibody-specific masked diffusion or flow matching model from scratch on OAS paired sequences, following the OADM objective [6] as in Zhao et al. This gives full control over the architecture and noise process (including germline-absorbing diffusion), but requires significant compute and careful data pipeline engineering.

Adapt EvoDiff [16]

EvoDiff is a publicly available discrete diffusion model for protein sequences trained on UniRef50. It uses the same OADM objective and can be fine-tuned on OAS antibody sequences. This reduces the compute burden substantially and provides a strong initialization.

For a conference submission on a university budget, fine-tuning EvoDiff on OAS is the more practical starting point. The scientific contribution lies in the scorer design and guidance mechanism, not in a novel generative architecture.

6.2 OAS data pipeline

The OAS database [3] is the standard training corpus for antibody language models. For the generative backbone, *unpaired* heavy and light chain sequences can be used (OAS contains hundreds of millions of these). For the SSH2.0 pseudo-labeling pipeline (Level 2), *paired* sequences are required; the paired OAS subset is smaller ($\sim$ tens of millions) but still orders of magnitude larger than GDPa.

The `oas_utils` package supports programmatic access to OAS subsets by species, study, and pairing status. A representative 100k–500k paired subset for pseudo-labeling requires approximately 2–5 GB of disk and can be downloaded in a few hours on a university network.

7 Pareto Optimization: Considerations for Future Work

The preceding framework uses a scalar composite scorer (Eq. (9)) for guidance. This implicitly encodes a fixed tradeoff between properties via the weights λ_i . A natural extension, deferred to future work or a second paper, is to replace scalar guidance with **multi-objective Pareto optimization**.

7.1 Why Pareto matters

The developability properties in GDPa are not independent: HIC and AC-SINS both capture hydrophobicity-related phenomena, while OASis humanness can trade off against certain developability improvements. A weighted sum obscures this structure and requires a prior commitment to a specific tradeoff that may not generalize. A Pareto front instead identifies all sequences where no property can be improved without degrading another, giving experimentalists a menu of candidates that spans the design space.

7.2 Integration points

Post-sampling filter (simple). Generate a large candidate pool via guided sampling, score each candidate on all objectives, and compute the Pareto non-dominated set using standard non-dominated sorting (NSGA-II style, $O(K^2m)$ for K candidates and m objectives). This requires no changes to the generative model and is straightforward to implement.

Decomposition-based parallel sampling (principled). Maintain K parallel sampling chains, each using a different scalarization vector $\lambda^{(k)}$ sampled from the m -simplex. At the end of sampling, select the Pareto non-dominated set from the K final sequences. This approach, inspired by MOEA/D [17], explicitly encourages the population to span the tradeoff space and is compatible with posterior tempering without architectural changes.

7.3 Uncertainty and the Pareto front

With scarce labeled data, the Pareto front computed over *predicted* objectives may contain spurious non-dominance relationships driven by scorer disagreement rather than genuine biophysical tradeoffs. The mitigation is to use the lower confidence bound from the ensemble scorer (Level 3) for each objective when computing the front: a sequence is considered Pareto non-dominated only if it is robustly good across the uncertainty range. This connects to the MESMO [18] framework for uncertainty-aware multi-objective Bayesian optimization.

7.4 Scientific value

Beyond practical utility, the Pareto framing answers a structural question about the developability landscape: are HIC and AC-SINS genuinely in tension in generated sequence space, or do sequences that score well on one tend to score well on both? If the Pareto front is thin and well-separated from the ideal point, the properties are in fundamental conflict. If it is close to the ideal point, a single well-designed sequence can satisfy all criteria simultaneously. This is an empirical question about antibody biophysics that the framework is uniquely positioned to answer.

8 Recommended Research Roadmap

Based on the considerations above, we propose the following phased approach toward a conference submission:

1. **Phase 1: Baseline replication.** Implement the masked discrete diffusion backbone (adapting EvoDiff or retraining on OAS) and the Level 0 scorer. Reproduce the key results of Zhao et al. under the discrete flow matching sampling scheme to confirm the baseline.
2. **Phase 2: Core contribution.** Implement the Level 2 SSH2.0 pseudo-label pipeline. Ablate against Level 0 to quantify the improvement from weak supervision. Implement the AbLang-2 naturalness regularizer (Level 4) and characterize the naturalness–developability tradeoff by sweeping α .
3. **Phase 3: Physics features.** Add PROPERMAB sequence features as auxiliary inputs to the GDPa regressor (Level 1, sequence-only variant). Report as an ablation.
4. **Phase 4: Uncertainty.** Wrap the regressor in a deep ensemble (Level 3) and evaluate whether LCB guidance reduces mode collapse relative to point-estimate guidance.
5. **Phase 5 (if bandwidth allows): Pareto.** Implement the post-sampling Pareto filter as a lightweight extension. Report the Pareto front geometry for HIC vs. AC-SINS as a characterization result.

References

References

- [1] S. Zhao, J. Moller, P. Quintero-Cadena, L. van Niekerk. *Guided Generation for Developable Antibodies*. arXiv:2507.02670, 2025.

- [2] A. Arsiwala et al. *GDPa: A Quantitative Developability Dataset for Paired Antibodies*. 2025.
- [3] T.H. Olsen et al. *Observed Antibody Space: A diverse database of cleaned, annotated, and translated unpaired and paired antibody sequences*. *Protein Science*, 2022.
- [4] X. Li et al. *Derivative-Free Guidance in Continuous and Discrete Diffusion Models with Soft Value-Based Decoding*. arXiv:2408.08252, 2024.
- [5] J. Austin et al. *Structured Denoising Diffusion Models in Discrete State-Spaces*. NeurIPS 2021.
- [6] E. Hoogetboom et al. *Argmax Flows and Multinomial Diffusion: Learning Categorical Distributions*. NeurIPS 2021.
- [7] J. Sanders et al. *Conditional generation of antibody sequences with classifier-guided germline-absorbing discrete diffusion*. arXiv:2605.06720, 2025.
- [8] Y. Lipman et al. *Flow Matching for Generative Modeling*. ICLR 2023.
- [9] A. Campbell et al. *Generative Flows on Discrete State-Spaces: Enabling Multimodal Flows with Applications to Protein Co-Design*. ICML 2024.
- [10] Z. Lin et al. *Evolutionary-scale prediction of atomic-level protein structure with a language model*. *Science*, 2023.
- [11] A. Raybould et al. *Five computational developability guidelines for therapeutic antibody profiling*. *PNAS*, 2019.
- [12] C. Abanades et al. *ImmuneBuilder: Deep-Learning models for predicting the structures of immune proteins*. *Communications Biology*, 2023.
- [13] PROPERMAB: an integrative framework for *in silico* prediction of antibody developability using machine learning. bioRxiv:2024.10.10.616558, 2024.
- [14] SSH2.0: A Better Tool for Predicting the Hydrophobic Interaction Risk of Monoclonal Antibody. *Frontiers in Genetics*, 2022. <http://i.uestc.edu.cn/SSH2/>
- [15] O. Olsen et al. *AbLang-2: A Foundation Model for Antibody Sequences*. 2023. `pip install ablang2`.
- [16] S. Alamdari et al. *Protein generation with evolutionary diffusion: sequence is all you need*. *bioRxiv*, 2023.
- [17] Q. Zhang, H. Li. *MOEA/D: A Multiobjective Evolutionary Algorithm Based on Decomposition*. *IEEE TEVC*, 2007.
- [18] S. Belakaria et al. *Max-value Entropy Search for Multi-Objective Bayesian Optimization*. NeurIPS 2019.